

Walmart Software Engineering Virtual Experience Program

Task 4: Database Population Script

Objective

The objective of this task is to populate the provided SQLite database using data from three CSV files:

- `shipping_data_0.csv`
- `shipping_data_1.csv`
- `shipping_data_2.csv`

The script performs the following operations:

1. Reads data from the CSV files.
2. Populates the `product` table with unique product names.
3. Processes shipment records from `shipping_data_0.csv`.
4. Aggregates shipment quantities from `shipping_data_1.csv`.
5. Merges shipment details from `shipping_data_2.csv`.
6. Inserts shipment records into the SQLite database.

Python Script

```
import pandas as pd
import sqlite3

# -----
# Connect to SQLite Database
# -----

conn = sqlite3.connect("shipment_database.db")
cursor = conn.cursor()
```

```

# -----
# Load CSV Files
# -----

df0 = pd.read_csv("shipping_data_0.csv")
df1 = pd.read_csv("shipping_data_1.csv")
df2 = pd.read_csv("shipping_data_2.csv")

# -----
# Populate Product Table
# -----

# Collect all unique product names from both datasets
products = pd.concat([
    df0["product"],
    df1["product"]
]).unique()

product_map = {}

# Insert products into database
for product in products:
    cursor.execute(
        "INSERT OR IGNORE INTO product (name) VALUES (?)",
        (product,)
    )

conn.commit()

# Create mapping between product name and product ID
cursor.execute("SELECT id, name FROM product")

for pid, pname in cursor.fetchall():
    product_map[pname] = pid

# -----
# Process shipping_data_0.csv
# -----

for _, row in df0.iterrows():

```

```

product_id = product_map[row["product"]]

cursor.execute("""
    INSERT INTO shipment
    (product_id, quantity, origin, destination)
    VALUES (?, ?, ?, ?)
""", (
    product_id,
    int(row["product_quantity"]),
    row["origin_warehouse"],
    row["destination_store"]
))

# -----
# Process shipping_data_1.csv and shipping_data_2.csv
# -----

# Calculate quantity of each product per shipment
grouped = (
    df1.groupby(
        ["shipment_idenfier", "product"]
    )
    .size()
    .reset_index(name="quantity")
)

# Merge shipment information
merged = grouped.merge(
    df2,
    on="shipment_idenfier"
)

# Insert merged shipment data
for _, row in merged.iterrows():

    product_id = product_map[row["product"]]

    cursor.execute("""
        INSERT INTO shipment
        (product_id, quantity, origin, destination)
        VALUES (?, ?, ?, ?)
    """)

```

```

        "", (
            product_id,
            int(row["quantity"]),
            row["origin_warehouse"],
            row["destination_store"]
        ))

# -----
# Commit Changes
# -----

conn.commit()

print("Database populated successfully!")

# -----
# Verification
# -----

cursor.execute("SELECT COUNT(*) FROM product")
print("Products:", cursor.fetchone()[0])

cursor.execute("SELECT COUNT(*) FROM shipment")
print("Shipments:", cursor.fetchone()[0])

conn.close()

```

Approach Summary

Product Table Population

All unique products from the provided CSV files were extracted and inserted into the **product** table. A mapping between product names and database IDs was created for later use.

Shipment Data Processing

Dataset 0

Each row already contained shipment quantity information. These records were directly inserted into the **shipment** table.

Datasets 1 and 2

- Dataset 1 contained one row per product item.
- Records were grouped by shipment identifier and product name to calculate quantities.
- Dataset 2 provided shipment origin, destination, and driver information.
- The grouped data was merged with Dataset 2 using the shipment identifier.
- The resulting shipment records were inserted into the database.

Result

The SQLite database was successfully populated with product and shipment information derived from all provided datasets.